



RESTful Jaxer

Jaxer, the Ajax server, can do a lot more than "just" serve pages and handle callbacks (function calls from your browser back to Jaxer). With Jaxer 1.0, you can easily create a variety of web services: RESTful, RPC-based, and almost anything else you can imagine. Hooks are already in place for a generic web service and for automatic RPC-based services. Here I'll demonstrate a simple RESTful service based on the generic web service hook.

You'll need [the latest Jaxer](#) (currently release 1.0) and the local_jaxer/conf folder contents that ship with it. If you've upgraded from a previous version of Jaxer or Aptana Studio (which has Jaxer built in) take a look at the Guide page for instructions on [updating your local_jaxer configuration to 1.0](#).

Ajax, meet server.

Ever wish you could use your Ajax, DOM, and Javascript skills on the server? Now you can!

Download Jaxer for your own server now!

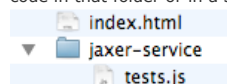
Ajax everywhere. **aptana Jaxer**

Here's a synopsis of all it takes to create a service (RESTful or otherwise):

1. create a folder called "jaxer-service"
2. put your code file in that folder (or anywhere under that folder if you need more hierarchy)

Your code is automatically loaded and executed whenever someone calls it. So now let's see how that works.

To offer a service through Jaxer, rather than a page, you'll need to put your code where the web server + Jaxer will recognize it as a service. By default, that means somewhere in your docroot where the path to it will contain "jaxer-service" or "jaxer-rpc". In a Studio project, create a folder with one of those names, and put your code in that folder or in a subfolder. For this example, I've created a very simple folder structure, one folder with two files:



This does two things: identifies the JavaScript code to Jaxer as a service, and prevents direct serving of your JavaScript file to a browser.

The service itself is just a JavaScript file that gets loaded (from disk or from cache) and executed every time the service is called. Your code has full access to the information in the request, and needs to set the response as you wish. Here I created a service that simply shows you much of the data available:

tests.js:

```
01. // pretty-print an object's properties:
02. function printObj(obj, indent)
03. {
04.     indent = indent || '';
05.     var lines = [];
06.     for (var p in obj)
07.     {
08.         if (typeof obj[p] == "function") continue;
09.         lines.push(indent + p + ": " + obj[p]);
10.     }
11.     return lines.join('\n');
12. }
13.
14. // in this sample, the HTTP status code to return is passed in the request:
15. var statusCodeToReturn = Jaxer.request.data.statusCode;
16.
17. // set the response contents to reflect the request information available:
18. Jaxer.response.setContents(
19.     printObj({
20.         'action (method)': '\n\t' + Jaxer.request.action,
21.         'resource': '\n\t' + Jaxer.request.parsedUrl.pathParts.slice(1).join("/"),
22.         'request headers': '\n\t' + printObj(Jaxer.request.headers, '\t'),
23.         'request body': '\n\t' + Jaxer.request.body,
24.         'data': '\n\t' + printObj(Jaxer.request.data, '\t'),
25.         'request': '\n\t' + printObj(Jaxer.request, '\t'),
26.         'environment variables': '\n\t' + printObj(Jaxer.request.env, '\t')
27.     }),
28.     statusCodeToReturn);
29.
30. // the Content-Type can be anything appropriate for your service:
31. Jaxer.response.headers['Content-Type'] = 'text/plain';
```

The service itself is done. (Of course you can now build an entire services layer, loading other files with Jaxer.load(...).) Now how do we call it? If e.g. the jaxer-service folder is directly under your docroot, just send a request to **/jaxer-service/tests** (and you can append whatever extra path or query items to this path).

How does this work? Jaxer ships with two default dispatchers, both in the extensions folder of the framework: serviceDispatcher.js and rpcDispatcher.js. The first one is what we'll use here (implicitly): it handles requests of the form /path/to/jaxer-service/path/to/myservice/any/additional/data that have jaxer-service in their path, and tries to find the corresponding file on disk. So it looks in your document root for /path.js, then for /path/to.js, then for /path/to/jaxer-service.js, and so on. In our case it will find /path/to/jaxer-service/path/to/myservice.js (or rather /jaxer-service/tests.js) and load it. How does the dispatcher get invoked? That's set in your configApps.js file, or in one of your other configApps-like peer files. By default Jaxer ships with services.configApps.js and that adds an app entry to Jaxer.Config.apps which knows to look for "jaxer-service" in the path and call Jaxer.Extensions.ServiceDispatcher.findFile(...); if it succeeds, the app entry has a "handler" property that returns the path to the file, which Jaxer then loads and executes. Note that this is all handled for you by the default configuration: all you have to do is follow the folder naming conventions. Of course, you can now modify this to suit your needs.

To test this service, I created a simple front-end page called index.html. The source is shown below. Here's what it looks like, including the output from the service:

RESTful Jaxer

What to send:

url:
 query params:
 HTTP action (method): ☐ GET ☐ POST ☒ PUT ☐ DELETE ☐ HEAD ☐ OPTIONS
 content type:
 request body:
 return code: (the response code Jaxer should return)

send

Response:

```

action (method):
  PUT
resource:
  tests
request
  headers:
    Host: 127.0.0.1:8000
    User-Agent: Mozilla/5.0 (Macintosh; U; Intel Mac OS X; en-US; rv:1.8.1.3) Gecko/20070322
    Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5
    Accept-Language: en-us,en;q=0.5
    Accept-Encoding: gzip,deflate
    Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
    Keep-Alive: 300
    Connection: keep-alive
    Cookie: JaxerSessionId%24%24127.0.0.1%3A8000=1223963136191015518628139664048
    Content-Type: application/x-www-form-urlencoded
    X-Requested-With: XMLHttpRequest
    Referer: http://127.0.0.1:8000/RESTfulJaxer/index.html
    Content-Length: 11
  request
    body:
      c=42&d=plum
  data:
    a: 12
    b: banana
    statusCode: 200
    c: 42
    d: plum
  request:
    constructorErrors:
      _request: [xpconnect wrapped aptIHttpRequest]
    method: PUT
    uri: /jaxer-service/tests?a=12&b=banana&statusCode=200
    protocol: HTTP/1.1
    remoteUser:
    remoteHost: 127.0.0.1
    remoteAddr: 127.0.0.1
    pageFile: /jaxer-service/tests
    documentRoot: /Users/sarid/Documents/Aptana Studio/RESTfulJaxer
    OS: Darwin
    isJaxerServer: true
    isHandler: true
    isHTTPS: false
    queryString: a=12&b=banana&statusCode=200
    logFileNativePath:
    action: PUT
    headers: [object Object]
    env: [object Object]
    current: [object Object]
    currentFolder: /jaxer-service
    bodyAsString: c=42&d=plum
    contentType: application/x-www-form-urlencoded
    contentLength: 11
    body: c=42&d=plum
    bodyAsByteArray: 99,61,52,50,38,100,61,112,108,117,109
    bodyAsStream: undefined
    data: [object Object]
    files:
    paramIsBody: false
    app: [object Object]
    appKey: 127.0.0.1:8000
    pageKey: 127.0.0.1:8000/jaxer-service/tests
    isCallback: false
    referer: [object Object]
    parsedUrl: [object Object]
    parsedUrlError:
  environment variables:
    CONTENT_LENGTH: 11
    GATEWAY_INTERFACE: CGI/1.1
    PATH_INFO:
    PATH_TRANSLATED: /jaxer-service/tests
    HTTP_HOST: 127.0.0.1:8000
    SCRIPT_NAME: /jaxer-service/tests
    SERVER_NAME: 127.0.0.1
    SERVER_PORT: 8000

```

You can change the method, see which ones support having a body and which do not, etc. Note that the calls are made using your browser's XHR (XMLHttpRequest) capabilities, wrapped in this simple example by the Jaxer client-side framework for convenience.

Note: if you're trying the above in Aptana Studio, it'll work out of the box. If you're trying it with the standalone Jaxer, remove the leading "/" in the URL of the service: use "jaxer-service/tests" instead. That's because Studio needs to do some "magic" remapping between the effective document root and the filesystem.

Here's the source for the front-end page:

index.html:

```

01. <html>
02.   <head>
03.     <title>RESTful Jaxer</title>
04.   </head>
05.   <body>
06.     <h1>RESTful Jaxer</h1>
07.     <script runat="server">Nothing - just embeds the Jaxer client framework</script>
08.     <h3>What to send:</h3>
09.     <form>
10.       <table>
11.         <tr><td>url:</td><td><input type="text" id="url" value="/jaxer-service/tests" size="30"></td></tr>
12.         <tr><td>query params:</td><td><input type="text" id="queryParams" value="a=12&b=banana" size="30"></td></tr>
13.         <tr><td>HTTP action (method):</td>
14.           <td>
15.             <input type="radio" name="httpAction" value="GET" checked="true">GET
16.             <input type="radio" name="httpAction" value="POST">POST
17.             <input type="radio" name="httpAction" value="PUT">PUT
18.             <input type="radio" name="httpAction" value="DELETE">DELETE
19.             <input type="radio" name="httpAction" value="HEAD">HEAD
20.             <input type="radio" name="httpAction" value="OPTIONS">OPTIONS
21.           </td>
22.         </tr>
23.         <tr><td>content type:</td><td><input type="text" id="contentType" value="application/x-www-form-urlencoded" size="30"></td></tr>
24.         <tr><td>request body:</td><td><input type="text" id="body" value="c=42&d=plum" size="30"></td></tr>
25.         <tr><td>return code:</td><td><input type="text" id="statusCode" value="200" size="5"> (the response code Jaxer should return)
26.         </td>
27.       </tr>
28.     </table>
29.   </form>
30.   <tr><td colspan="2"><input type="button" value="send" onclick="send()"></td></tr>
31.   <hr>
32.   <h3>Response:</h3>
33.   <pre id="result" style="margin: 0pt"></pre>
34.   <script>
35.     function $(id) { return document.getElementById(id); }
36.     function $$ (id) { return $(id).value; }
37.     function getRadio(form, name)
38.     {
39.       var elts = form.elements[name];
40.       for (var i=0; i<elts.length; i++)
41.       {
42.         if (elts[i].checked) return elts[i].value;
43.       }
44.     }
45.     function send()
46.     {
47.       var result = Jaxer.XHR.send($$('body'),
48.       {
49.         url: $('url') + '?' + $('queryParams') + "&statusCode=" + $('statusCode'),
50.         method: getRadio(document.forms[0], 'httpAction'),
51.         cacheBuster: false,
52.         async: false,
53.         contentType: $('contentType')
54.       });
55.       $('result').innerHTML = result;
56.     }
57.   </script>
58. </body>
59. </html>

```

To summarize, all we needed to do to create the service was to create a JavaScript file somewhere in the document root where its path would contain "jaxer-service". The file will be loaded and executed whenever a request is received for that path (without the ".js" extension). You can add other special paths by adding them to your web server configuration — see the *.httpd.conf files that ship by default with the standalone Jaxer, and to configApps.js and its peers.

In a future blog I'll also show an example of an RPC (Remote Procedure Call) service. Jaxer can also easily consume RESTful services server-side, using e.g. Jaxer.Web.get(...), Jaxer.Web.post(...), Jaxer.Web.put(...), Jaxer.Web.del(...), and Jaxer.Web.head(...) — note that the DELETE action is not invoked via delete(...) because "delete" is a reserved identifier in JavaScript.